



EDU TANTR

SERVERLESS IMAGE RESIZER

AWS-Based Automated Image Resizer

CLOUD COMPUTING INTERNSHIP PROJECT REPORT

Prepared By

RAKSHITA S K S

Cloud Computing Intern

EDU TANTR

Mentor

KUMAR D

INTERNSHIP OVERVIEW

This project was developed as part of my Cloud Computing Internship at Edutantr using Amazon Web Services (AWS). The project provided practical experience in serverless computing, cloud storage, event-driven processing and image processing using AWS services.

DETAIL	INFORMATION
Internship Domain	Cloud Computing
Project Title	Serverless Image Resizer
Cloud Platform	Amazon Web Services (AWS)
Project Type	Serverless Cloud Application
Role	Cloud Computing Intern

TABLE OF CONTENTS

S.NO.	CONTENT	PAGE
1	INTRODUCTION	1
2	PROBLEM STATEMENT	2
3	OBJECTIVES	2
4	AWS SERVICES AND TECHNOLOGIES USED	3
5	SYSTEM ARCHITECTURE	4
6	SYSTEM WORKFLOW	5
7	IMPLEMENTATION	6
8	CONFIGURATION AND INTEGRATION	8
9	TESTING AND RESULTS	10
10	OUTPUT	13
11	CHALLENGES AND SOLUTION	16
12	KEY LEARNINGS	17
13	CONCLUSION	18

1.INTRODUCTION

The Serverless Image Resizer was developed as part of my Cloud Computing Internship at Edutantr using Amazon Web Services (AWS). The main purpose of this project was to automatically resize an image whenever it was uploaded to an Amazon S3 bucket.

I configured an S3 bucket with an ObjectCreated trigger connected to an AWS Lambda function. The Lambda function was developed using Python and the Pillow library to process the uploaded image. After processing, the resized image was stored in the required S3 location.

Through this project, I gained practical experience in Amazon S3, AWS Lambda, IAM permissions, Python-based image processing and Amazon CloudWatch monitoring.

2.PROBLEM STATEMENT

Resizing images manually for different requirements can take additional time and effort. A cloud-based automated solution can process an image immediately after it is uploaded.

To address this, I developed a Serverless Image Resizer using Amazon S3 and AWS Lambda. The system automatically triggers the Lambda function when an image is uploaded, processes the image using Python and Pillow, and stores the resized image in the S3 bucket.

3.OBJECTIVES

- To create an automated image resizing system using AWS.
- To store the input image in Amazon S3.
- To trigger AWS Lambda automatically when an image is uploaded.
- To process the image using Python and Pillow.
- To store the resized image in the required S3 location.
- To configure IAM permissions for the Lambda function.
- To monitor Lambda execution using Amazon CloudWatch.
- To verify the final resized image.

4.AWS SERVICES AND TECHNOLOGIES USED

SERVICE/ TECHNOLOGY	PURPOSE	PRACTICAL USE
Amazon S3	Cloud storage	Stored the input and resized images
AWS IAM	Access control	Provided permissions to Lambda
AWS Lambda	Serverless processing	Automatically processed the uploaded image
Amazon CloudWatch	Monitoring	Checked Lambda execution logs
Pillow (PIL)	Image processing	Enabled server-side encryption for stored objects
Python	Programming	Opened and resized the image

5.SYSTEM ARCHITECTURE

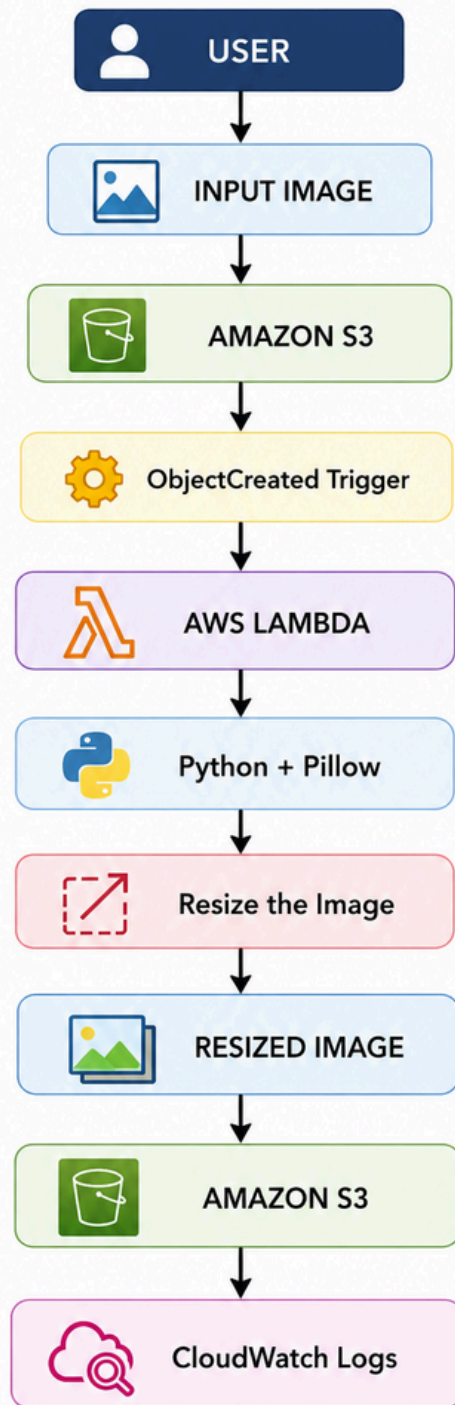


Figure 5.1: Architecture of the Serverless Image Resizer

6.SYSTEM WORKFLOW

The Serverless Image Resizer was developed using AWS cloud services to automatically process uploaded images, resize them using AWS Lambda, store the output in Amazon S3, and monitor the configured resources.

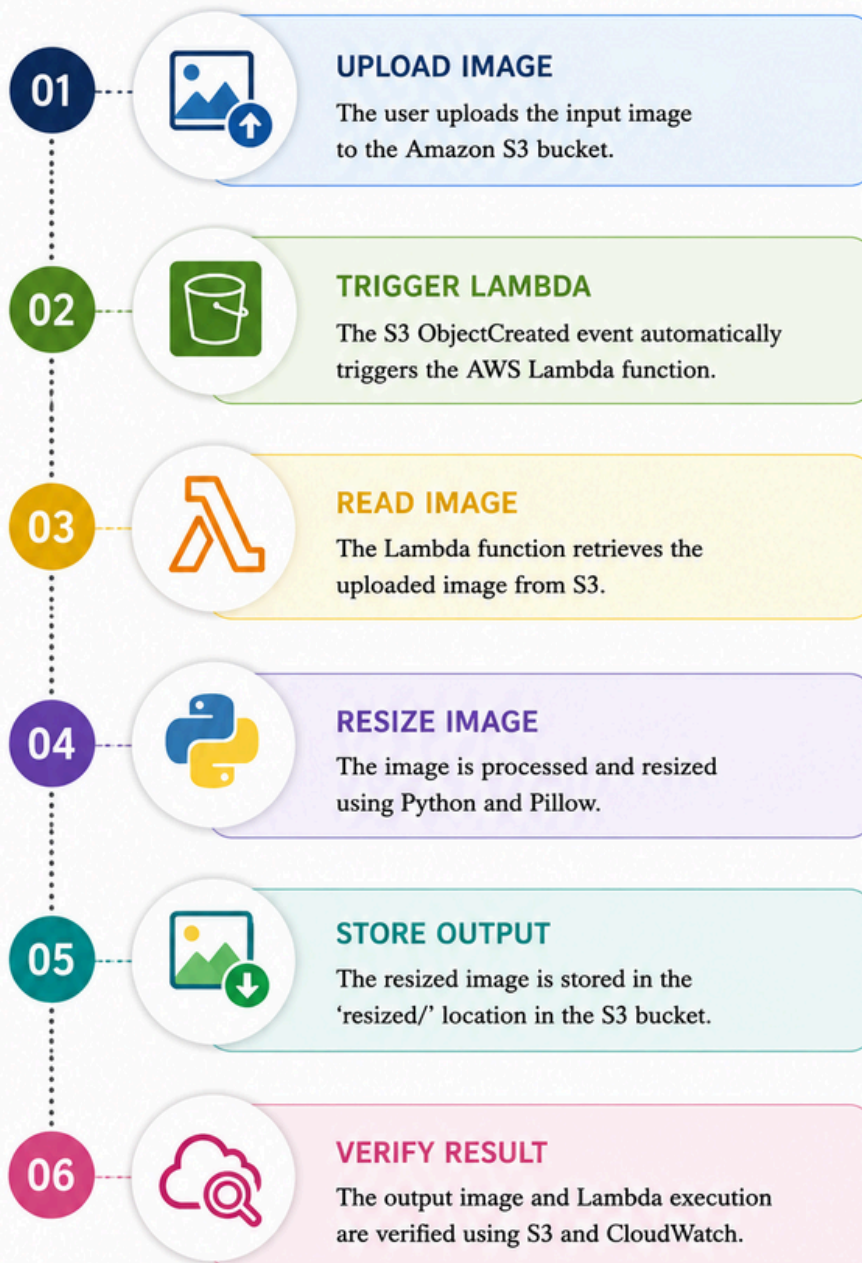


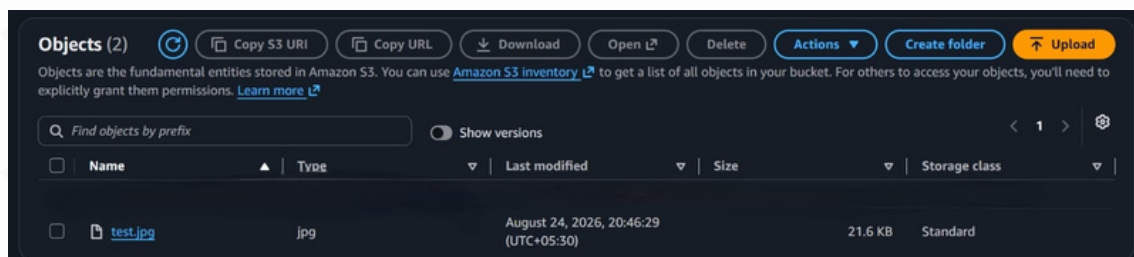
Figure 6.1: System Workflow of the Image Resizer

7.IMPLEMENTATION

The Serverless Image Resizer was implemented by configuring Amazon S3, AWS Lambda, IAM permissions and the required Python image-processing dependency. The services were integrated using an S3 ObjectCreated trigger.

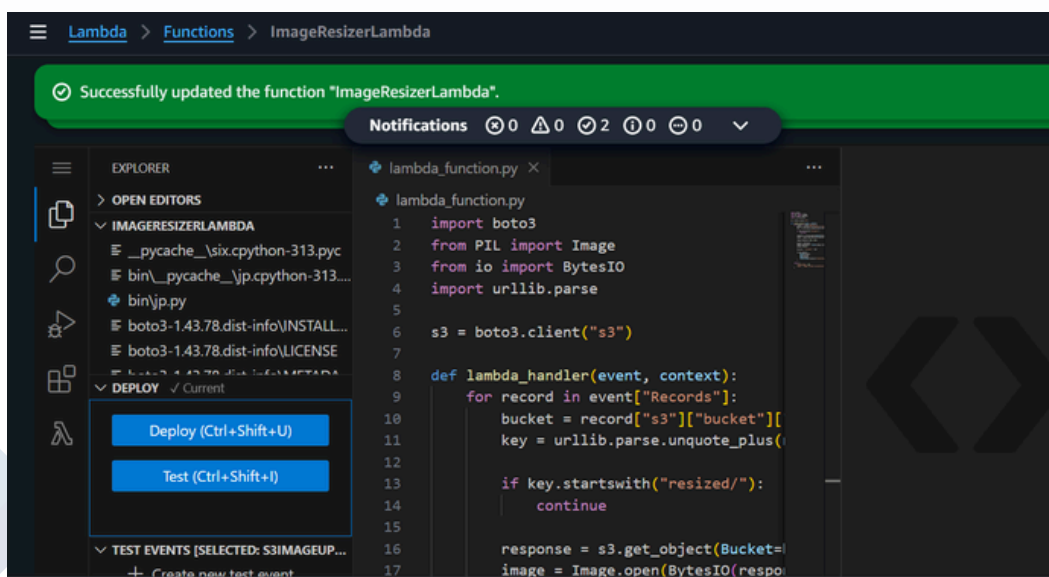
7.1.AMAZON S3 CONFIGURATION

An Amazon S3 bucket was configured to store the input image. The test image was uploaded to the bucket before triggering the image processing workflow.



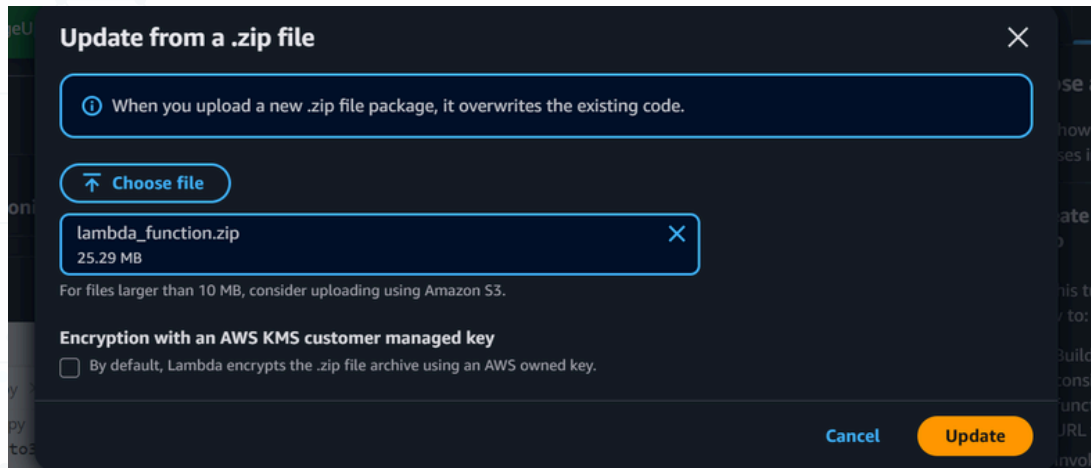
7.2.AWS LAMBDA CONFIGURATION

An AWS Lambda function was created using the Python runtime. The function was configured to receive the S3 event and process the uploaded image.



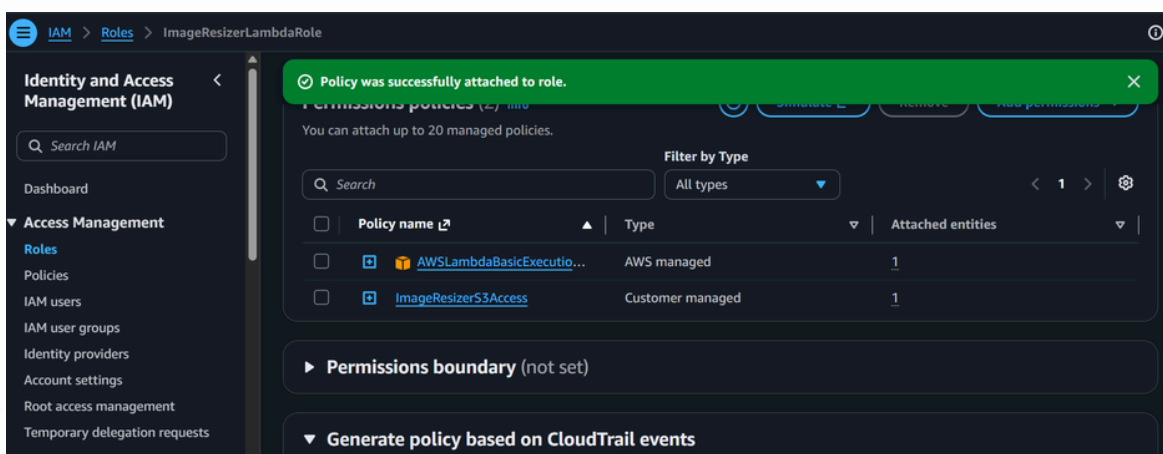
7.3.PILLOW DEPENDENCY

The Pillow library was included with the Lambda deployment package to provide the required image processing functionality.



7.4.IAM EXECUTION ROLE

An IAM execution role was configured to provide the Lambda function with the required permissions to access the S3 resources and write execution logs.

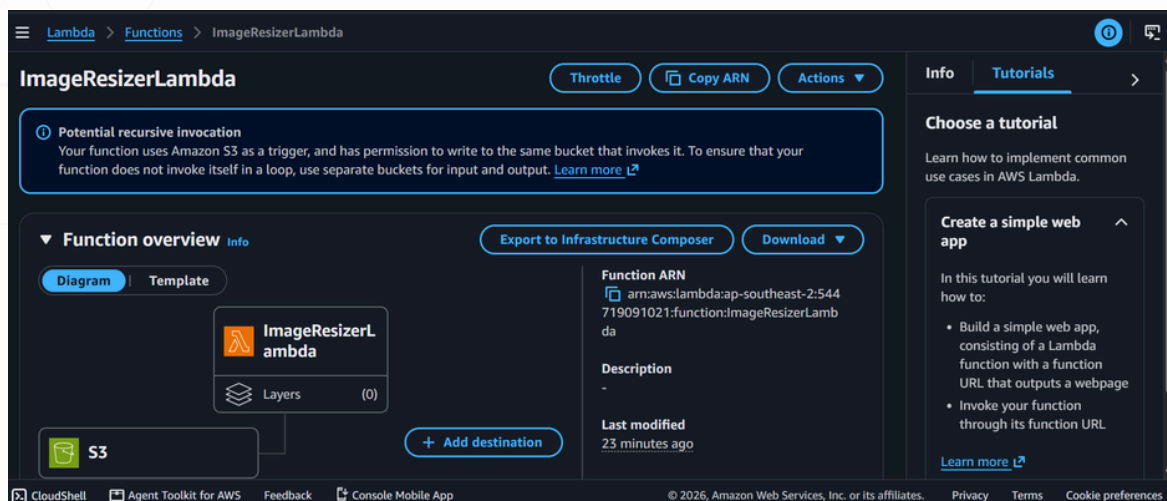


8.CONFIGURATION AND INTEGRATION

The AWS services were configured and integrated to automate image processing, connect the S3 upload event with Lambda, process the image using Python and Pillow, and store the resized output in Amazon S3.

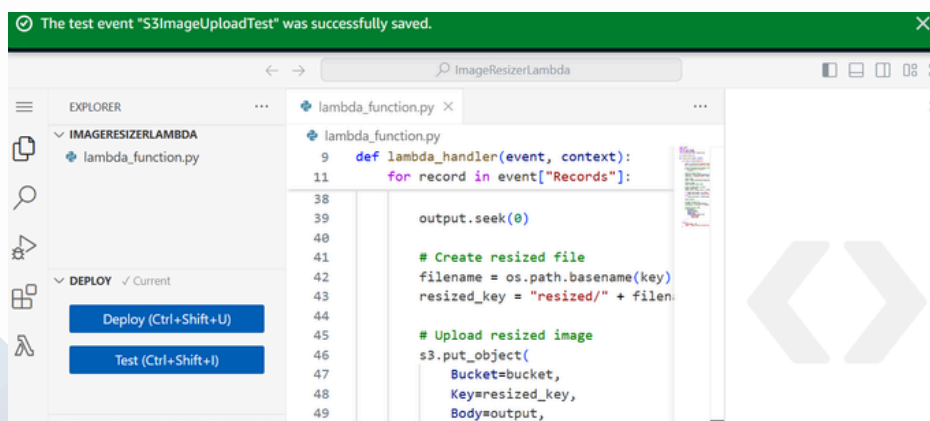
8.1.S3 LAMBDA TRIGGER

An S3 ObjectCreated trigger was configured to automatically invoke the Lambda function whenever an image was uploaded to the bucket.



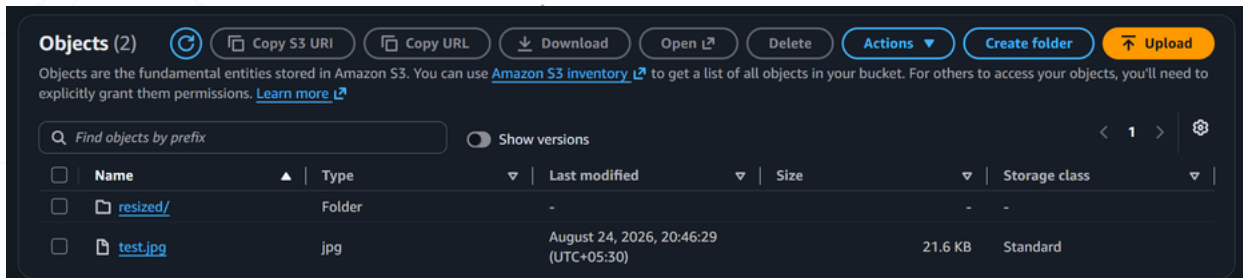
8.2.LAMBDA PROCESSING

The Lambda function receives the S3 event, retrieves the uploaded image and processes it using Python and Pillow.



8.3.OUTPUT LOCATION

After processing, the resized image was stored in the designated resized/ location in Amazon S3.



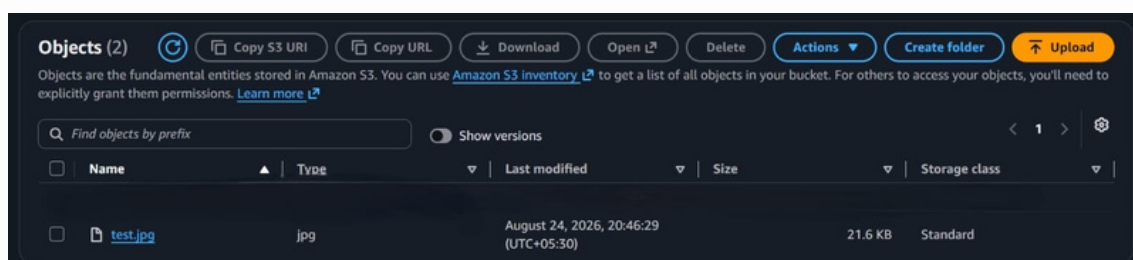
9.TESTING AND RESULTS

After completing the configuration, the system was tested by uploading the test.jpg image to Amazon S3. The S3 event triggered the Lambda function automatically, and the image was processed using the configured Python and Pillow logic.

The Lambda execution was checked to verify successful processing, and the generated output was then verified in the S3 resized/ location.

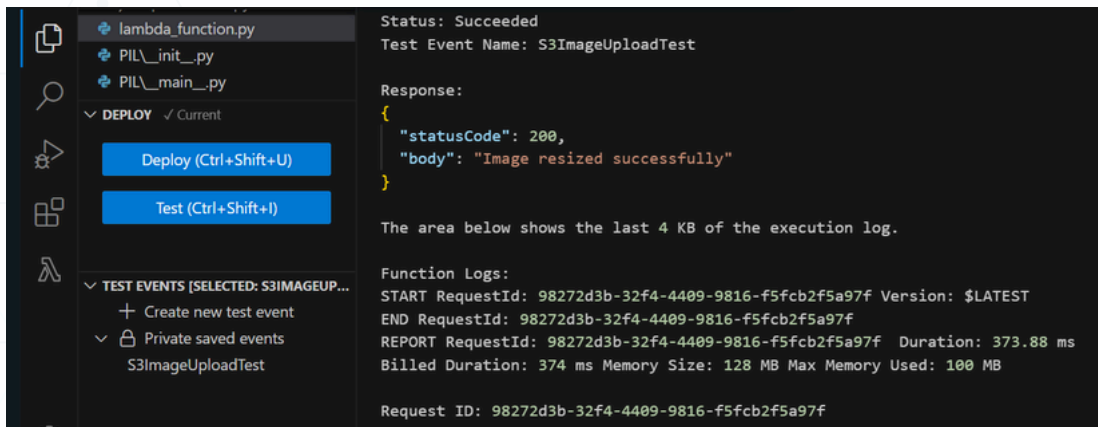
9.1.INPUT VERIFICATION

The original test.jpg image was successfully uploaded to the S3 bucket.



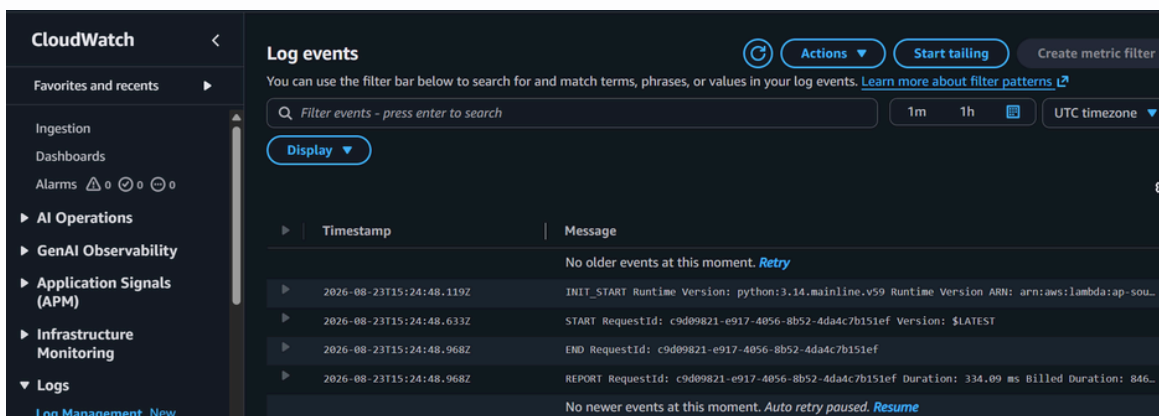
9.2.LAMBDA EXECUTION

The Lambda test execution completed successfully and confirmed that the image processing function executed correctly.



9.3.CLOUDWATCH MONITORING

The Lambda execution activity was monitored using Amazon CloudWatch to verify the execution and check the generated logs.



RESULT

The test was successful. The uploaded test.jpg image was processed and the resized output was generated and stored in the S3 resized/ location.

10.OUTPUT

The final output of the Serverless Image Resizer is the automatically generated resized image stored in Amazon S3. The output was verified after successful Lambda execution.

10.1.S3 ORIGINAL IMAGE

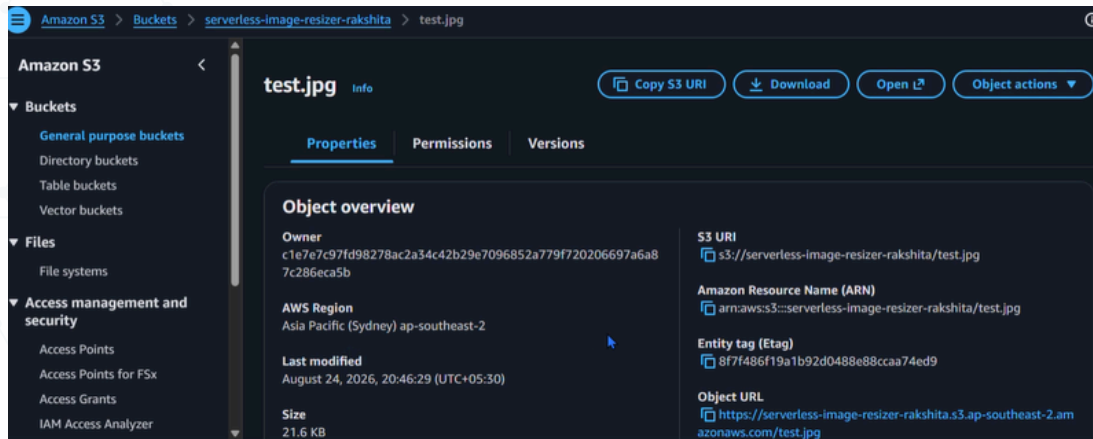


Figure 10.1: Original image uploaded to Amazon S3

10.2.RESIZED OUTPUT

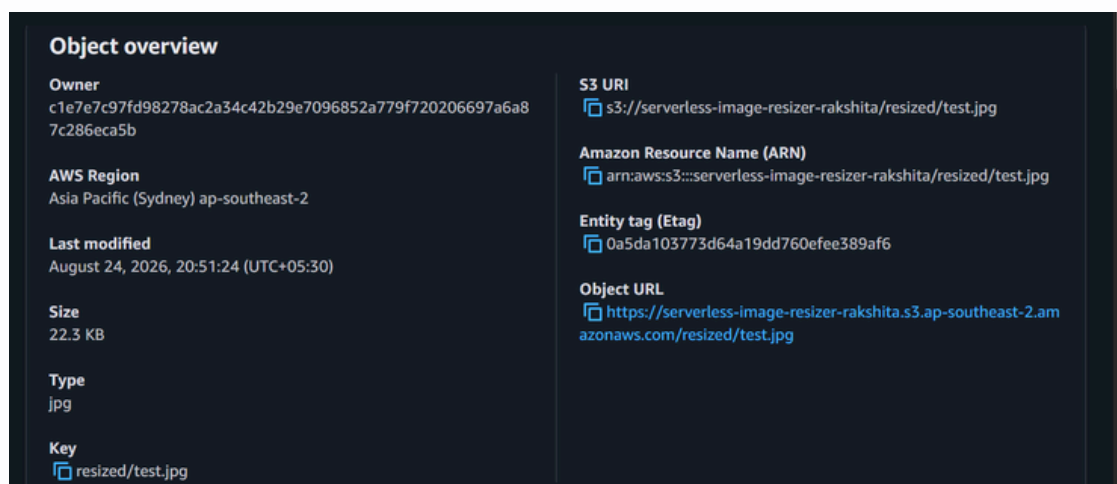


Figure 10.2:Resized image generated and stored in Amazon S3

10.3.LAMBDA+S3 TRIGGER

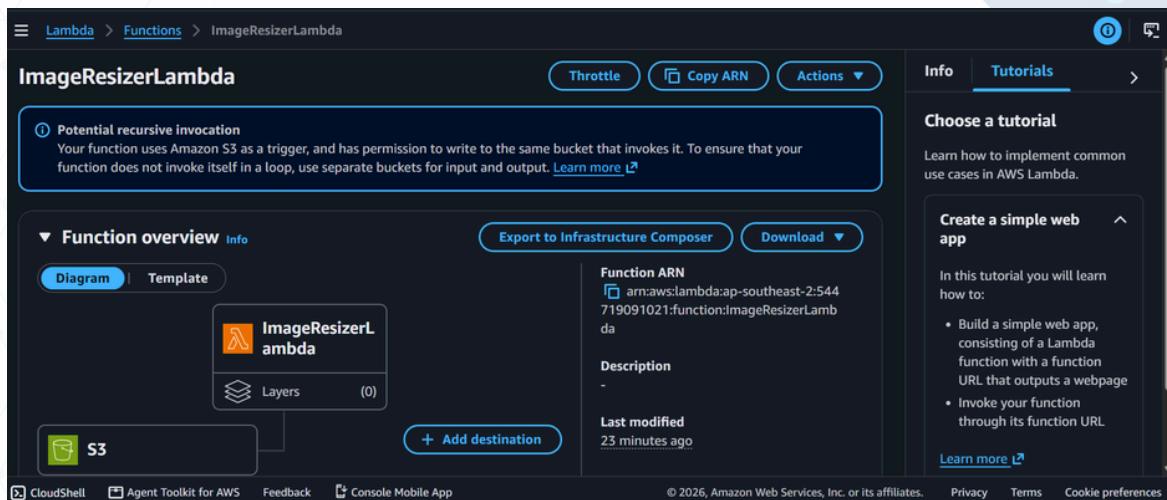


Figure 10.3:S3 trigger configured to invoke the AWS Lambda function.

10.4.SUCCESSFUL LAMBDA FUNCTION

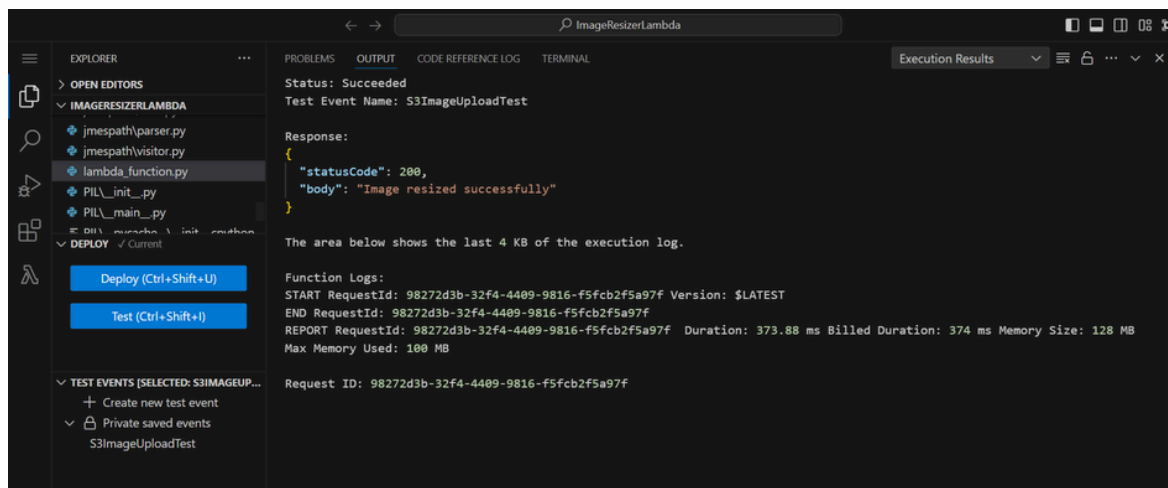


Figure 10.4:Successful execution of the image resizing Lambda function

10.5.CLOUDWATCH LOGS

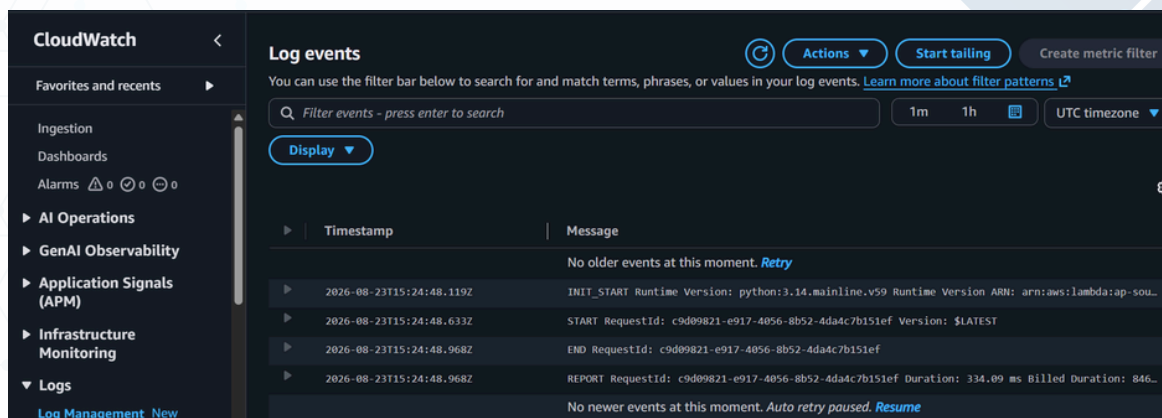
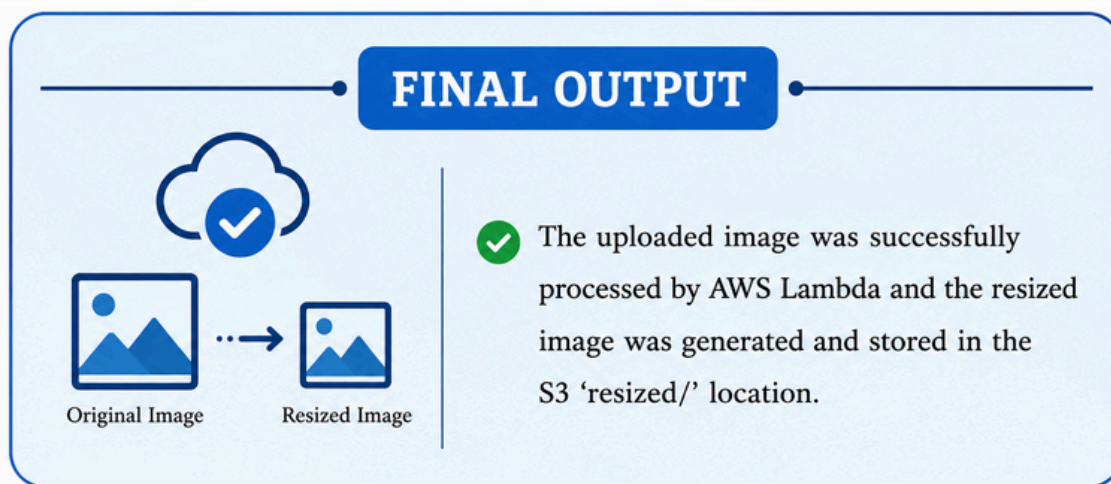


Figure 10.5: AWS CloudWatch logs showing Lambda execution activity



11. CHALLENGES AND SOLUTION

CHALLENGE FACED	SOLUTION
Configuring the S3 trigger with Lambda	Configured the correct S3 trigger and tested the Lambda function.
Packaging the Pillow dependency	Packaged Pillow with the Lambda deployment package.
Configuring IAM permissions	Configured the required Lambda execution role and permissions.
S3 object key mismatch during testing	Checked the actual S3 object key and corrected the test event.
Verifying Lambda execution	Used CloudWatch logs to monitor and verify Lambda execution.

12.KEY LEARNINGS

Through this project, I gained practical experience in building an event-driven serverless application using AWS.

- Learned to create and manage Amazon S3 buckets and objects.
- Learned to configure S3 ObjectCreated triggers.
- Gained practical experience in creating AWS Lambda functions.
- Learned to use Python and Pillow for image processing.
- Learned to configure IAM execution roles and permissions.
- Learned to monitor Lambda execution using CloudWatch.
- Improved my ability to troubleshoot AWS configuration and testing errors.
- Understood how S3 and Lambda can work together for automated processing.

12.1.REFERENCES

1. Amazon Web Services — Amazon S3 Documentation
2. Amazon Web Services — AWS Lambda Documentation
3. Amazon Web Services — AWS IAM Documentation
4. Amazon Web Services — Amazon CloudWatch Documentation
5. Pillow Documentation

13.CONCLUSION

The Serverless Image Resizer provided me with practical experience in developing an event-driven cloud application using AWS. I configured Amazon S3 to store the input image, connected it with AWS Lambda using an S3 trigger, and used Python with Pillow to process the image.

The resized image was successfully generated and stored in the S3 resized/ location. I also configured IAM permissions and used Amazon CloudWatch to monitor the Lambda execution.

Overall, this project improved my practical knowledge of Amazon S3, AWS Lambda, IAM, CloudWatch, Python, Pillow and serverless event-driven processing.